

Finnid HIS

Hospital Information System — ERP

Multi-Tenant SaaS Platform for Hospitals

Complete End-to-End Process Guide

From Project Start → New Hospital Onboarding → Patient Journey → Billing → Discharge

Demo Presentation

Prepared: 01 July 2026

Contents

- 1. Project Overview — what Finnid HIS is
- 2. Architecture at a Glance
- 3. Part A — Start the Project (run & sign in)
- 4. Part B — Add a NEW Hospital (onboarding)
- 5. Part C — Configure the Hospital (users, roles, modules)
- 6. Part D — End-to-End Patient Journey → Billing
- 7. Part E — Real-time & Smart Features
- 8. Part F — Compliance, Audit & Security
- 9. Part G — Year-End (Fiscal-Year Shift) & Decommission
- 10. Suggested Live Demo Flow (15 minutes)
- 11. Demo Credentials & URLs (quick reference)
- 12. Quality & Verification (CI + tests)
- 13. Status — Live vs Pending

1. Project Overview

Finnid HIS is a multi-tenant SaaS Hospital ERP. One running platform serves many hospitals; each hospital (a “tenant”) gets its own databases, its own fiscal-year financial data, its own web domain, and role-scoped logins — all provisioned automatically in seconds, with zero manual database work.

Key idea in one line:

- Add a hospital in ~2 seconds; it instantly has patient registration, clinical records, diagnostics, pharmacy, billing, insurance/scheme claims, HR/payroll, real-time boards, AI assists and statutory compliance — all isolated from every other hospital.

Who uses it:

Role	Who	What they do
Superadmin	Platform owner (you)	Onboards hospitals, manages modules/roles, runs year-shifts. Belongs to NO hospital.
Tenant Admin	A hospital's own admin	Manages that hospital's staff logins (create/edit/roles/reset). Scoped to its own hospital only.
Hospital Staff	Doctors, nurses, billing, pharmacy...	Do daily work; the menu shows only what their role is granted.

2. Architecture at a Glance

Three database planes per platform — the heart of the design:

Plane	Database	Holds
Control plane	HIS_Platform (one, shared)	List of tenants, fiscal years, domains, RBAC, billing, audit.
Longitudinal	{CODE}_Master (per hospital)	Patients, clinical history, doctors, wards/beds, masters. Never splits — history stays continuous across years.
Financial	{CODE}_FY{YYYY-YY} (per year)	Bills, payments, claims, pharmacy, payroll, diagnostics — one DB per fiscal year.

Why two DBs per hospital: clinical history must stay continuous (it never splits), while money (bills/claims/payroll) is fiscal-year-scoped. So patient/clinical → _Master, financial → _FY{year}.

Technology stack:

Layer	Technology
Backend	ASP.NET Core (.NET 9), CQRS + MediatR, Dapper, FluentValidation
Security	JWT auth, PBKDF2 password hashing, MFA/TOTP, AES-256-GCM field encryption at rest, RBAC
Real-time	SignalR hubs (queue board, emergency alerts, ambulance GPS) — isolated per tenant
Interop / AI	FHIR R4 export; transparent rule-based AI (risk, forecast, claim pre-scrub)
Data	MS SQL Server (LocalDB in dev, Azure SQL in prod); routing via platform.DbCatalog + host→tenant
DevOps	GitHub Actions CI (build + functional smoke), Docker, Azure Container Apps

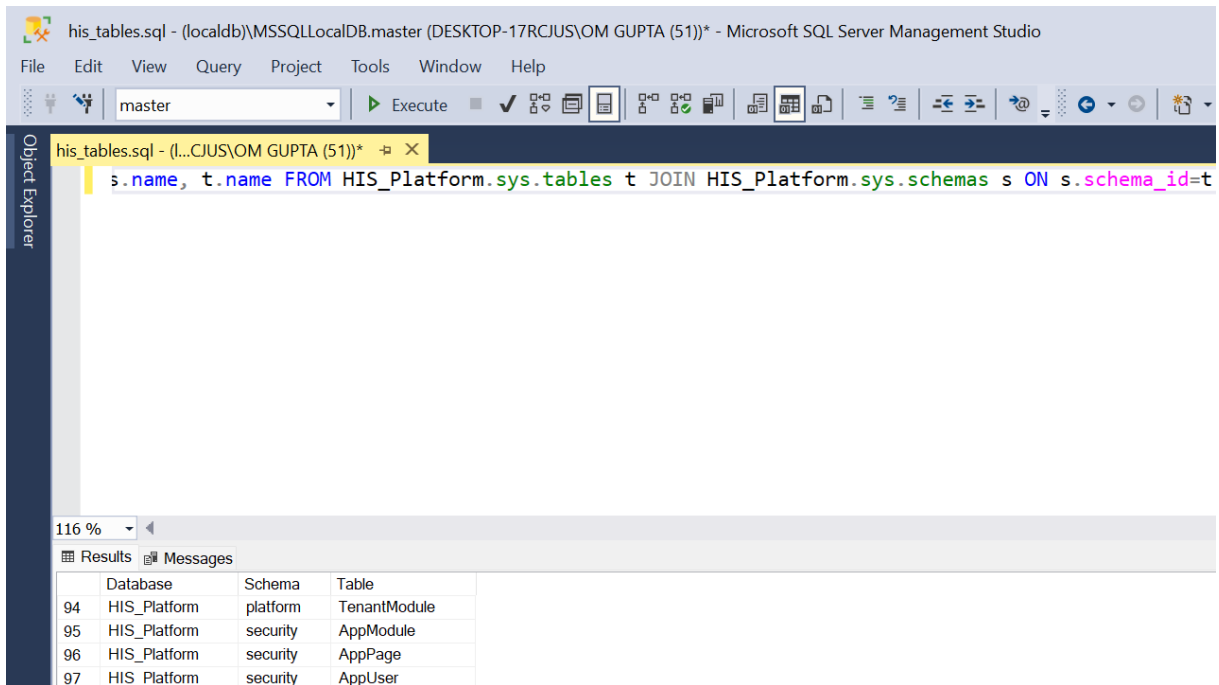


Fig 1. The database planes in SQL Server — HIS_Platform (control) plus per-tenant Master and fiscal-year (FY) tables.

3. Part A — Start the Project (run & sign in)

One-time platform setup (developer / first run):

1. Prerequisites

.NET 9 SDK, MS SQL (LocalDB or a server), sqlcmd, Node 18+ (for tests).

2. Create the control-plane database + seed it

Run the platform migrations — this creates HIS_Platform (the control plane).

3. Run the API (it also serves the web UI)

dotnet run --project src/HIS.Api → open the printed URL, e.g. <http://localhost:5142>. On first boot the app auto-creates the superadmin and a demo DEV tenant.

4. Sign in as Superadmin

Go to the login page and sign in with the superadmin credentials (see §11). The title bar shows a “Platform Admin” link → opens the console.

What you see after login: a landing page, the clinical Workspace (left sidebar of modules scoped to your role), and — for the superadmin — the Platform Admin console.

4. Part B — Add a NEW Hospital (Onboarding)

Done by the Superadmin, in the browser, in ~2 seconds, with zero manual DB work.

Steps:

1. Open the wizard

Platform Admin → Onboard hospital.

2. Fill the form

Field	Example	Meaning
Tenant code	ACME	Short id (letters/digits/-). Becomes the DB-name prefix.
Hospital name	ACME Multispeciality	Display name.
Fiscal year	FY 2026-27	Indian FY (Apr→Mar) by default; configurable.
Primary domain	acme.hospital.in	The hospital's own URL.
Common-domain alias	acme.app.finnid.in	(Optional) shared-domain access.

3. Click “Provision & onboard”

What happens automatically (POST /api/platform/tenants/onboard) — verified end-to-end:

- **Step 1:** Provisions the databases first: creates {CODE}_Master and {CODE}_FY{YYYY-YY}, applies the schema-split templates, seeds reference masters.
- **Step 2:** Registers the tenant, the fiscal year (start/end), and the domains (primary + common) for request routing.
- **Step 3:** Registers the DBs in platform.DbCatalog (the routing source of truth).
- **Step 4:** Enables all modules for that tenant × fiscal year.
- **Step 5:** Writes billing: a per-FY Subscription + a ledger charge.
- **Step 6:** Audits the action. If any step fails, it rolls back — no orphan tenant/DBs.

Verify: the Tenants table lists the hospital with its {CODE}_Master + {CODE}_FY... databases; both exist in SQL Server with the schema split (master, patient, clinical, billing, insurance...).

5. Part C — Configure the Hospital

Task	How	Status
Create the hospital's login users	Platform Admin → Users: create (tenant + id + password + name + role); per-user Edit, Change role, Reset password, Deactivate/Activate. Each user is bound to its tenant and can log in only to that hospital.	Live
Tenant-admin self-service	A hospital's own admin sees a “Manage Users” link — the same Users screen, locked to its own tenant (enforced server-side from the JWT).	Live
Assign modules/pages to roles (RBAC)	Platform Admin → Modules & Roles → grant module / page / action to a role. The sidebar + APIs scope to these grants.	Live
Enable/disable a module per fiscal year	Platform Admin → Entitlements → toggle a module for tenant × FY.	Live
Point the domain	DNS: point acme.hospital.in at the app. The platform resolves host → tenant automatically.	Ops

5. Nursing & diet

Record nursing notes (vitals) and a diet order against the admission.

6. Discharge → housekeeping

Discharge frees the bed (→ Cleaning). Housekeeping “Mark ready” returns the bed to the available pool (clean → free) so it recycles for the next patient.

6.3 Diagnostics & Pharmacy

7. Lab order → results

Module: Lab. Order investigations → barcode LB-FY2026-27-000007 → enter results.

8. Radiology / Blood bank

Raise radiology orders; check blood stock and raise a blood request.

9. Pharmacy dispense

Module: Pharmacy. Dispense against the prescription from batch/stock (batch-tracked, FEFO).

6.4 Billing & Payment

10. Create bill (from tariff master)

Module: Billing. Add charge lines from the tariff master → Bill BILL-FY2026-27-000008 with computed gross, discount, insurance and net payable.

11. Collect payment

Module: Payments. Collect via the configured payment gateway (e.g. Razorpay) → receipt.

6.5 Insurance / Cashless / Government Schemes

12. Capture policy + eligibility

Module: Insurance/Cashless. Capture the patient's policy; check eligibility.

13. Pre-authorisation

Raise a pre-auth → Claim CL-FY2026-27-000001 (e.g. PreAuth ₹50,000 → Approved ₹40,000).

14. Claim lifecycle

The claim moves PreAuth → Approval → Settlement, each stage recorded as an immutable ClaimEvent.

15. Settlement reconciliation

On payment, bank UTR is reconciled against the approved amount → status Matched; the claim is marked Settled.

16. Government schemes

PM-JAY (BIS verify + HBP-rate TMS claim), ESIC / CGHS verification — same claim engine.

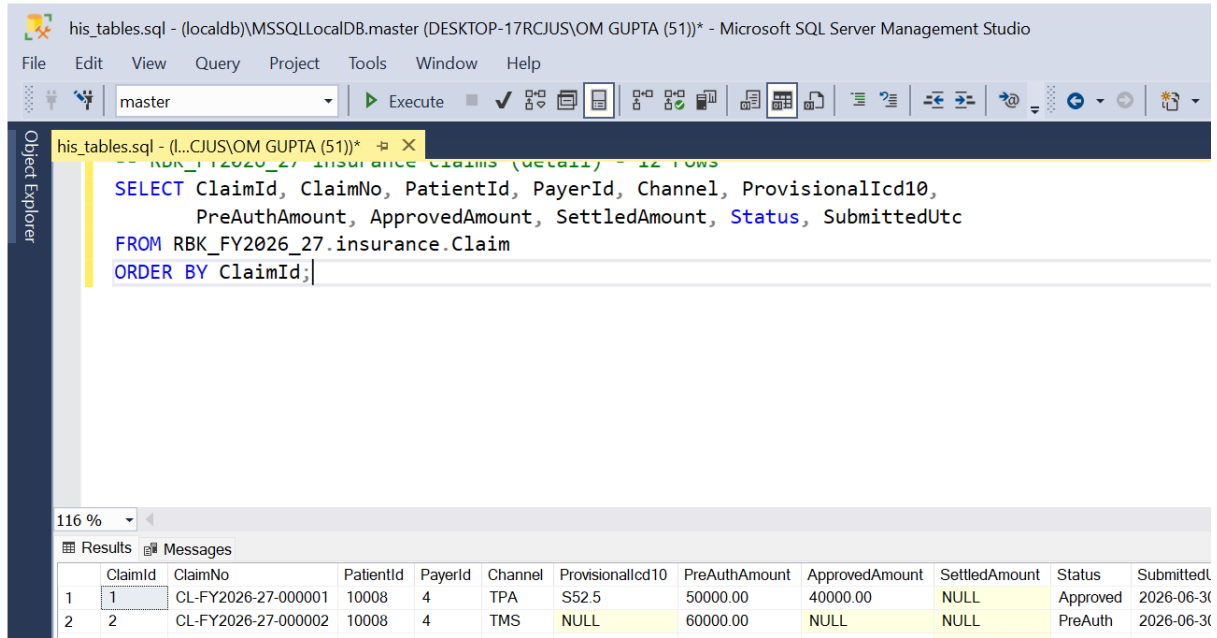


Fig 3. Insurance claims with fiscal-year claim numbers (CL-FY2026-27-...), channel (TPA/TMS), amounts and lifecycle status.

6.6 Close the visit

17. Discharge summary & certificates

Issue the discharge summary and any statutory certificates (fitness, MLC, etc.).

Result: a complete, auditable episode — Register → Appointment → Consult/Admit → Diagnostics/Pharmacy → Bill & Pay → Claim & Settle → Discharge — all inside the correct tenant DB and fiscal year.

7. Part E — Real-time & Smart Features

Feature	What it does
Live Queue Board	OPD queue counters + tokens update live (SignalR), isolated per tenant.
Emergency Alert Hub	Broadcasts critical/ED alerts in real time to the right tenant only.
Ambulance GPS Tracking	Dispatch ambulances and stream live GPS pings to a tracking board.
AI Suite	Patient risk (NEWS2-style score/band), inventory reorder forecast, and claim pre-scrub (auto-reject over-package before submission). Transparent, rule-based, config-swappable.
FHIR R4 Export	Exposes patients/records as FHIR R4 resources for interoperability.

8. Part F — Compliance, Audit & Security

- **Audit:** Immutable audit trail — every command is recorded (who/what/when), both control-plane and per-tenant.
- **Access:** RBAC — permissions gate every action; the menu itself is scoped to a role's grants.

- **Security:** MFA/TOTP + PBKDF2 password hashing + AES-256-GCM encryption of sensitive fields at rest.
- **Statutory:** Bio-Medical Waste (BMWM colour-coded bags + Form-IV), MLC (auto-numbered), consent capture.
- **Isolation:** Per-tenant database isolation and host→tenant routing — hospitals never see each other's data.

9. Part G — Year-End (Fiscal-Year Shift) & Decommission

Year shift (no downtime, history intact):

1. Open next fiscal year

Platform Admin → Open next fiscal year → enter tenant code + pick next FY → Open.

2. What happens

Provisions the next {CODE}_FY{next} data DB; carries module entitlements forward; rolls billing forward (CarryForward of prior balance + new charge). Clinical/patient data stays in the same {CODE}_Master — it never splits.

Decommission a tenant (cleanup):

Platform Admin → Tenants → red “Decommission” card. Enter + re-type the tenant code to confirm. It drops all the tenant's databases first, then deletes its control-plane rows (the immutable audit trail is retained). Tenant admins cannot do this.

10. Suggested Live Demo Flow (15 minutes)

A tight click-by-click order for the presentation:

Time	Do this	Notes
0:00	Landing + sign in as Superadmin	Show the Platform Admin console.
2:00	Onboard a new hospital “ACME”	Fill wizard → Provision. Then show the two new databases (ACME_Master, ACME_FY2026-27) in SSMS / the Tenants list.
4:00	Create a hospital user + role	Users panel → create an admin/doctor; show it is scoped to ACME.
5:30	Switch to the Workspace (DEV demo tenant)	Register a patient → UHID; book an appointment/token.
7:30	OPD consult → IPD admit	Diagnosis; admit to a bed on the live Bed Board; nursing note; discharge; Mark bed ready.
9:30	Diagnostics + Pharmacy	Lab order + result; pharmacy dispense.
11:00	Billing → Payment	Create a bill from tariff; collect payment.
12:30	Insurance claim lifecycle	Policy → pre-auth → approval → settlement (Matched).
13:30	Show the smart layer	AI risk/forecast/pre-scrub; live Queue board; Compliance & Audit trail; FHIR export.
14:30	Wrap: quality	Show GitHub Actions green + the smoke suite (67 automated functional tests).

11. Demo Credentials & URLs (quick reference)

Item	Value
App URL (dev)	http://localhost:5142
Landing page	/ (marketing landing)
Workspace (clinical)	/app/index.html
Platform Admin console	/app/admin.html
Login page	/app/login.html
Superadmin login	superadmin / ChangeMe!2026 (dev bootstrap — change in production)
Demo tenant	DEV (auto-provisioned on first boot; fiscal year FY2026-27)

Note: dev passwords are placeholders. Production supplies the JWT key / bootstrap password / connection strings via environment or Azure Key Vault.

12. Quality & Verification

- **CI:** GitHub Actions CI on every push: build + a deep functional smoke suite that boots the API against a real SQL Server and exercises every module over HTTP.
- **Tests:** 67 automated functional tests pass (auth, registration, OPD/IPD, diagnostics, pharmacy, billing, payments, insurance, schemes, HR/payroll, occ-health, telemedicine, BMW, MLC, queue, feedback, AI, dashboard, audit, FHIR, bed housekeeping). 0 failures.
- **Scope:** Remaining items are external integrations (UIDAI/ABDM/NHCX/payment-gateway) and infra (load/DR) — intentionally mocked/pending, not defects.

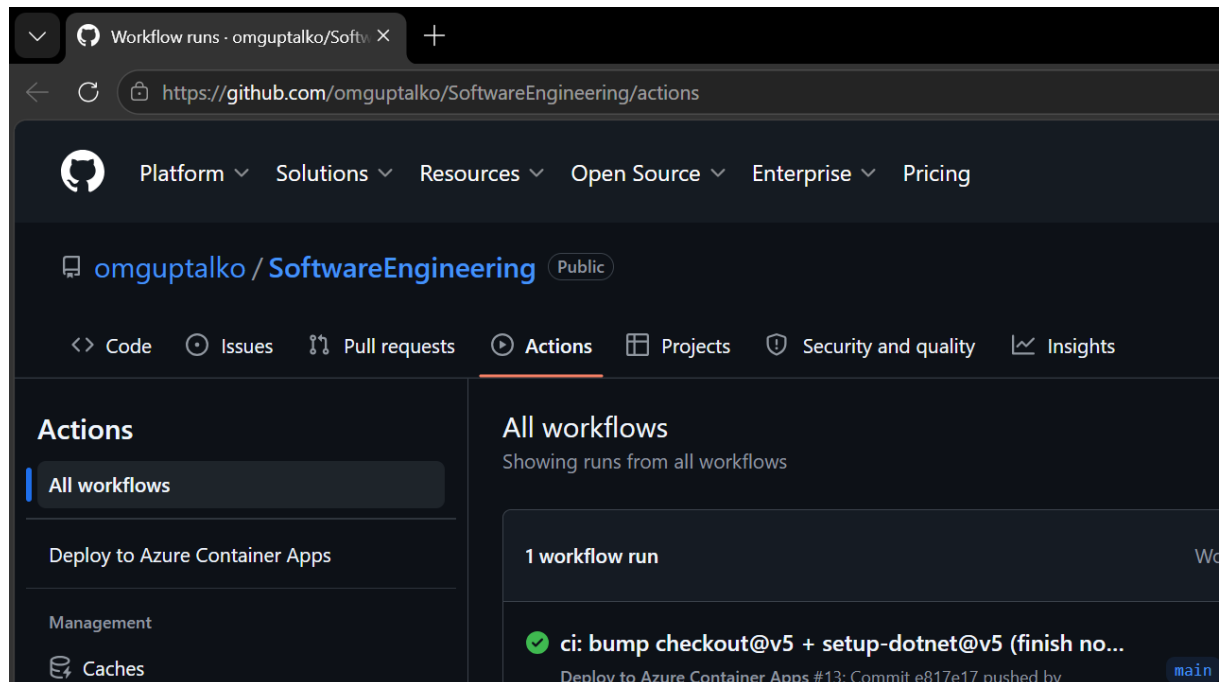


Fig 4. GitHub Actions — the CI pipeline (build + functional smoke) passing green on the latest commit.

	A	B	C	D	E	F	G	H	I	J	K	L	M
1	Module	Metric	DEV_FY20	RBK_FY20	RBK_FY20: Match								
2	Insurance	Claims (to	12	12	12 MATCH								
3	Insurance	Claims Set	6	6	6 MATCH								
4	Insurance	Claims Pre	6	6	6 MATCH								
5	Insurance	Reconcilia	6	6	6 MATCH								
6	Insurance	Total Settl	240000	240000	240000 MATCH								
7	Billing	Bills	7	7	7 MATCH								
8	Billing	Payments	6	6	6 MATCH								
9	Diagnostic	Lab Order	6	6	6 MATCH								
10	Diaenostic	Lab Result	6	6	6 MATCH								

Fig 5. Cross-database parity check — module row-counts identical across the fiscal-year databases (all MATCH).

13. Status — Live vs Pending

Live & verified:

Automated onboarding (DBs / schemas / domains / billing / modules), year-shift, per-tenant DB isolation, domain→tenant routing, RBAC-scoped menu, MFA + AES-at-rest, immutable audit trail, the full clinical + financial module set, real-time boards (queue / alerts / GPS), AI assists (risk / forecast / pre-scrub), FHIR R4 export, and the closed bed-lifecycle (admit → discharge → housekeeping).

Pending (before a real hospital go-live):

Real external integrations (UIDAI / ABDM / NHCX / PM-JAY / ESIC / CGHS / payment gateways) and real Azure-ML AI — currently scaffolded/mockied; production TLS + secret sourcing (Key Vault); and a DR/backup plan.

— End of Guide —